LP04 BOUCLES FOR ET WHILE

Les boucles FOR

La boucle `FOR` est une boucle bornée. On sait à l'avance combien de fois on va la faire.

```
# En pseudo code
instruction1
Pour i allant de 0 a 6:
    instruction2
instruction3
```

- instruction1 est effectuée
- `i` est un `itérateur` qui va aller de 0 à 6, c'est à dire 7 tours
- On effectuera donc 7 fois l'instruction2
- Quand on a fini la boucle on effectue l'instruction3

La boucle FOR en python

Code Python

```python
for i in range(0, 7):
    print(i, end=" ")
# affichage:
# 0 1 2 3 4 5 6
```

Code Python

```python
for i in range(2, 7):
    print(i, end=" ")
# affichage:
# 2 3 4 5 6
```

Code Python

```python
for i in range(5):
    print(i, end=" ")
# affichage:
# 0 1 2 3 4
```

Prototypage de la boucle FOR

La boucle FOR s'écrit de manière générale de la façon suivante :

Code Python

```python
for i in range(a, b):
```

La boucle est effectuée $b-a$ fois et la variable `i` prend pour valeur tous les entiers entre $[a;b[$

Exemple : Algorigramme de la structure FOR

`for i in range(2, 9):`

Code Python

```python
for i in range(2, 9):
    print(i)
print('fin')
```

L'itérateur `i` prend toutes les valeurs entières entre 2 et 9 – 1

i	i < 9	instruction
2	True	print(2)
3	True	print(3)
4	True	print(4)
5	True	print(5)
6	True	print(6)
7	True	print(2)
8	True	print(2)
9	False	print('fin')

Itération sur une chaine de caractères via les indices

Une chaine de caractères `str` est un itérateur, c'est à dire qu'il est possible d'itérer sur ses éléments.

```
>>> mot = 'Hello'
>>> mot[1]
'e'
>>> mot[3]
'l'
```

Indice	0	1	2	3	4
Caractère	H	e	l	l	o
Indice Négatif	-5	-4	-3	-2	-1

Ainsi si on fait varier un itérateur `i`, il est possible d'obtenir chaque élément de `mot` en appelant `mot[i]`

Code Python

```python
mot = 'Hello'
for i in range(0, len(mot)):
    print(mot[i])
print('Fin')
```

i	i < len(mot)	instruction	affichage
0	True	print(mot[0])	H
1	True	print(mot[1])	e
2	True	print(mot[2])	l
3	True	print(mot[3])	l
4	True	print(mot[4])	o
5	False	print('Fin')	Fin

Itération sur une chaine de caractères via les éléments

Une chaine de caractères `str` est un itérateur, c'est à dire qu'il est possible d'itérer directement sur les éléments qui constituent cette chaine de caractère
On peut donc créer un variable nommée `lettre` par exemple qui prendra pour valeur chacun des caractères de `mot`.

Code Python

```python
mot = 'Hello'
for lettre in mot:
    print(lettre)
print('Fin')
```

lettre	instruction	affichage
'H'	print(lettre)	H
'e'	print(lettre)	e
'l'	print(lettre)	l
'l'	print(lettre)	l
'o'	print(lettre)	o
None	print('Fin')	Fin

- Il n'y a pas besoin de connaitre la longueur du mot
- Il n'y plus l'utilisation d'indice
- `lettre` prend pour valeur chaque caractère de mot
- lorsque `lettre` a pris pour valeur tous les caractères de `mot`, la boucle s'arrête.

Exercice 1

Écrire une fonction permettant d'afficher tous les entiers entre 0 et 6 inclus

Exercice 2

Écrire une fonction permettant d'afficher tous les entiers entre 2 et 9 inclus

Exercice 3

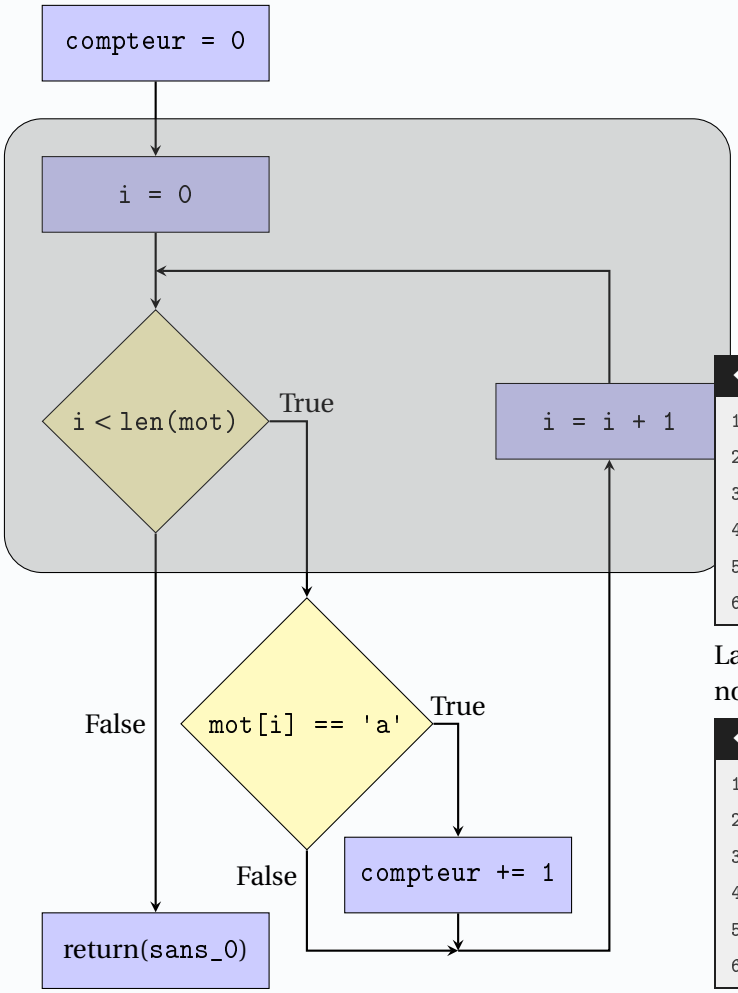
Écrire une fonction permettant d'afficher toutes les lettres du mot `'abracadabra'` en itérant sur les indices

Exercice 4

Écrire une fonction permettant d'afficher toutes les lettres du mot `'abracadabra'` en itérant sur les caractères

Algorithmes de Recherche dans un itérable

Puisqu'on peut itérer sur une chaine de caractère en observant chaque caractère, on peut décider de faire quelque chose si la caractère a ue particularité.
Ce algorithme compte le nombre de caractères `'a'` dans la chaine de caractère `mot`



- on initialise un compteur à 0
- on parcourt `mot` à l'aide d'une boucle `FOR`
- on observe donc chaque caractère `mot[i]`
- si le caractère est un `'a'`, alors on incrémente le compteur
- sinon on ne fait rien
- dans tous les cas on refait une boucle

Code Python

```python
def compte_a(mot):
    compteur = 0
    for i in range(len(mot)):
        if mot[i] == 'a':
            compteur=compteur+1
    return compteur
```

La version suivante permet de compter le nombre de `caractere` dans `mot`

Code Python

```python
def compte(mot, caractere):
    compteur = 0
    for i in range(len(mot)):
        if mot[i] == caractere:
            compteur=compteur+1
    return compteur
```

Exercice 5

En vous inspirant de du programme précédent, proposer un programme permettant de compter le nombre de `lettre_cherchee` dans `mot` en itérant sur les éléments.

Code Python

```python
def compte_lettre_cherchee(mot):
    #
```

Exercice 6

Écrire une fonction `est_present(mot, lettre_cherchee)` qui renvoie `True` si `lettre_cherchee` est dans `mot` et `False` sinon

Code Python

```python
def est_present(mot, lettre_cherchee):
    #
```

I. La boucle WHILE

Les boucles WHILE

La boucle `WHILE` est une boucle non bornée. On ne sait à l'avance combien de fois on va la faire.

```
# En pseudo code
instruction1
tant que condition1 vraie:
    instruction2
instruction3
```

- instruction1 est effectuée
- tant que la condition1 est vraie on effectue l'instruction 2
- quand la condition1 est fausse, on sort de la boucle ;
- on effectue l'instruction3

La boucle WHILE en python

Code Python

```python
a = 0
while a < 5:
    print(a, end=" ")
    a = a + 1
# affichage: 0 1 2 3 4
```

Code Python

```python
lettre, mot = 'd', 'abracadabra'
i = 0
trouve = False
while not trouve:
    trouve = (lettre == mot[i])
    i = i + 1
print(i)
# affichage 7
# !!! possibilité de boucle infinie
```

Prototypage de la boucle WHILE

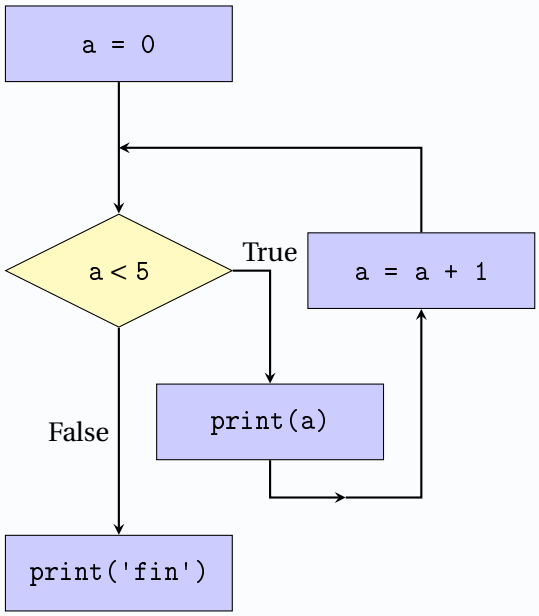
La boucle WHILE s'écrit de manière générale de la façon suivante :

Code Python

```python
while condition1:
    instruction
```

La boucle est effectuée tant que `condition1` est vraie. Cette condition est donc un booléen

Algorigramme de la structure WHILE



Code Python

```python
a = 0
while a < 5:
    print(a, end=" ")
    a = a + 1
print('fin')
```

a	a < 5	instruction
0	True	print(0)
1	True	print(1)
2	True	print(2)
3	True	print(3)
4	True	print(4)
5	False	print('fin')

Exercice 7

Boucle While

1. On dispose d'une feuille de papier d'épaisseur 0,1 mm. Combien de fois doit-on la plier au minimum pour que l'épaisseur dépasse la hauteur de la tour Eiffel 324 m. Écrire une fonction `nb_plis()` en Python pour résoudre ce problème.

   Modifier ce programme pour passer en argument la hauteur à atteindre.

2. Aujourd'hui un appartement vaut 100 000€. Sa valeur augmente de 1
   - Écrire un programme en Python pour connaitre sa valeur au bout de 10 ans.
   - Écrire un programme en Python pour savoir au bout de combien d'années sa valeur aura doublé.

3. Inès veut construire une pyramide à base carrée comme sur la photo. La pyramide sur la photo a 7 étages.

   - Inès a 1000 billes. Combien d'étages au maximum aura sa pyramide ? Écrire un programme en Python pour répondre au problème.
   - Modifier ce programme pour déterminer combien d'étages on obtient avec n billes.

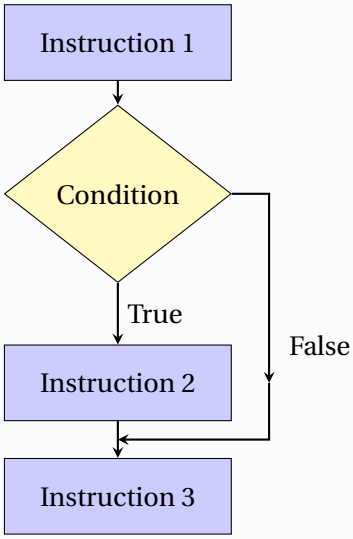
4. Le 1er janvier, les parents de Gaspard ont déposé sur son livret d'épargne 1500 euros au taux d'intéret de 1,5% pour qu'il s'achète un scooter à 1700 euros.
   - Écrire un programme en Python pour savoir au bout de combien d'années il pourra avoir son scooter
   - Modifier le programme pour que l'utilisateur puisse choisir le nombre d'années et la somme initiale.
   - Modifier le programme pour que l'utilisateur puisse également choisir le taux d'intéret en pourcentage.

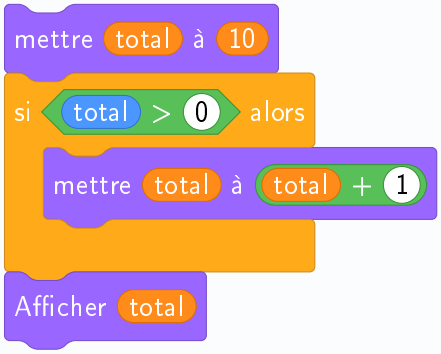
5. L'ordinateur tire un nombre entier au hasard entre 0 et 100. L'utilisateur doit le trouver et pour cela propose des valeurs. L'ordinateur indique pour chaque valeur proposée si la valeur est trop petite, trop grande ou s'il a trouvé.
   - Écrire un programme en Python pour jouer à ce jeu. En combien de coups est-on sûr de trouver ?
   - Modifier le programme pour qu'il s'arrête si l'utilisateur n'a pas trouvé au bout du nombre de coups de la question 2.

6. En mathématiques, la suite de Fibonacci est une suite d'entiers dans laquelle chaque terme est la somme des deux termes qui le précèdent. Elle commence généralement par les termes 0 et 1 (parfois 1 et 1) et ses premiers termes sont 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, etc.
   - Ecrire un programme qui affiche les termes de la suite de Fibonacci jusqu'à ce qu'un des termes soit supérieur strictement à 1000.
   - Modifier ce programme pour obtenir les termes de la suite jusqu'à ce qu'un des termes dépasse n qui sera passé en paramètre.

7. L'utilisateur donne un entier positif et le programme annonce combien de fois de suite cet entier est divisible par 2.
8. Écrire une fonction qui fait des tirages successifs d'entiers 0 ou 1 jusqu'à tirer 3 fois 1 consécutivement et renvoie le nombre de tirages qui ont été nécessaires.
9. Ecrire une fonction python qui renvoie l'entier p tel que $2^p \leqslant n \leqslant 2^{p+1}$.

Les structures conditionnelles IF






```python
# En python
Instruction_1
if condition:
    Instruction_2
Instruction_3
```

```python
# En python
total = 10
if total > 0:
    total = total + 1
print(total)
```

- l'`Instruction 1` est effectuée
- SI la `Condition` est `True` alors l'`Instruction 2` est effectuée
- SINON on va directement à l'`Instruction 3`
- Dans tous les cas l'`Instruction 3` est effectuée

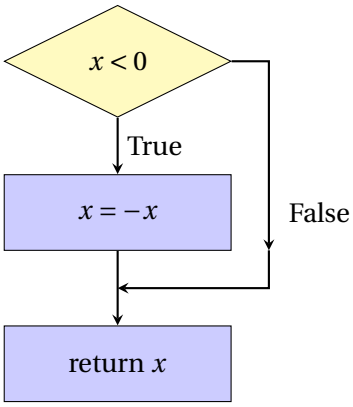
La `Condition` est un test qui est évalué en python par un booléen.

Exemple : La condition IF sans ELSE : la valeur absolue d'un nombre

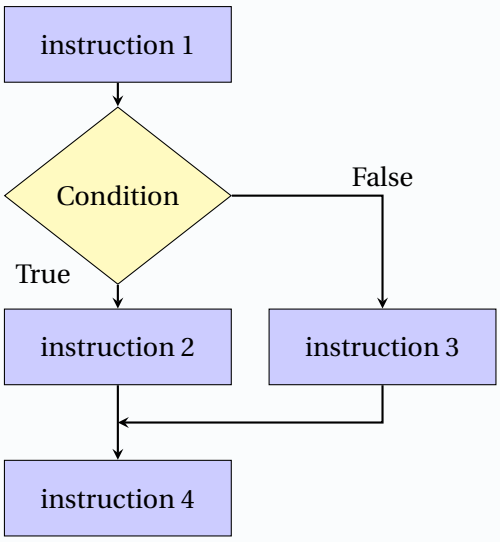
- si $x \geqslant 0$, alors $|x| = x$
- si $x < 0$, alors $|x| = -x$

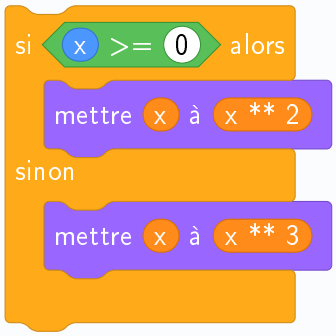
Code Python

```python
def valeur_absolue(x):
    ''' Renvoie |x| '''
    if x < 0:
    # si x < 0 on échange x par -x
        x = - x
    return x
```



Les structures conditionnelles IF – ELSE






```python
# En python

instruction_1

if condition:
    instruction_2
else:
    instruction_3
instruction_4
```

- l'`instruction 1` est effectuée
- SI la `Condition` est `True` alors l'`instruction 2` est effectuée
- SINON l'`instruction 3` est effectuée
- Dans tous les cas l'`instruction 4` est effectuée

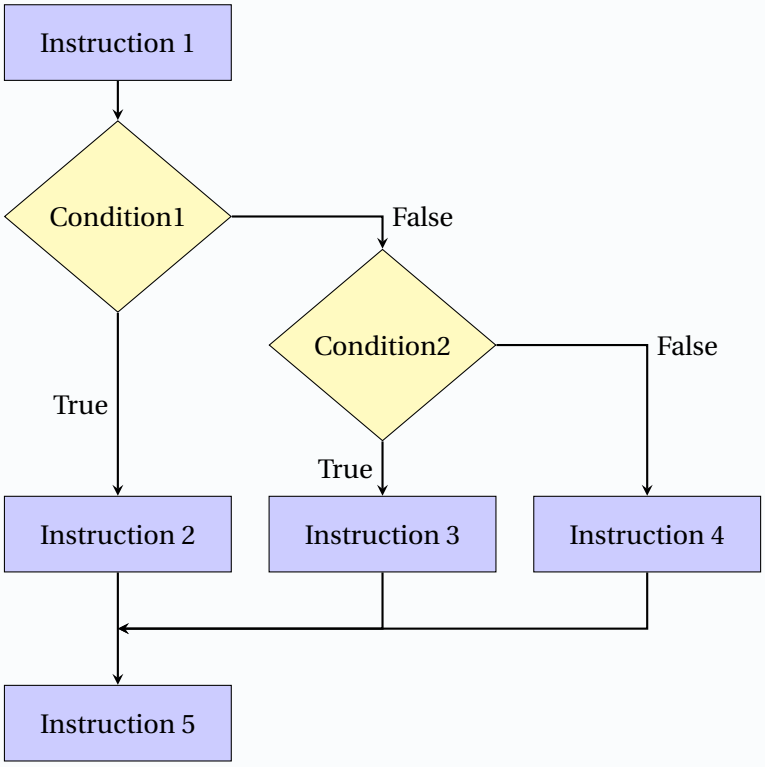
Exemple : La condition IF – ELSE

Code Python
```python
def ma_fonction(x):
    if x >= 0:
        sortie = x ** 2
    else:
        sortie = x ** 3
    return sortie
```

Les structures conditionnelles IF – ELIF – ELSE

Dans le cas où il y a plusieurs possibilités de réponse, on peut utiliser deux structures différentes en python. On suppose une fonction prenant comme paramètre une variable x de type entier.

$$f(x) = 1 \text{ si } x > 0 \qquad f(x) = 0 \text{ si } x = 0 \qquad f(x) = -1 \text{ si } x < 0$$



Code Python
```python
def ma_fonction(x):
    if x > 0:
        sortie = 1
    elif x == 0:
        sortie = 0
    else:
        sortie = -1
    return sortie
```

Code Python
```python
def ma_fonction(x):
    if x > 0:
        sortie = 1
    else:
        if x == 0:
            sortie = 0
        else:
            sortie = -1
    return sortie
```

Exercice 8 : Compléter les fonctions suivantes

Code Python
```python
def plus_grand2(a, b):
    ''' Renvoie le plus grand des
    deux nombres a et b '''
    # A compléter



    #
```

Code Python
```python
def plus_grand3(a, b, c):
    ''' Renvoie le plus grand des
    trois nombres a, b et c '''
    # A compléter



    #
```